

ОТЧЁТ

по лабораторной работе №2

«Параллельное умножение квадратных матриц с
использованием OpenMP»

Никишин Владислав

Группа: 6312

Содержание

1	Цель работы	2
2	Постановка задачи	2
3	Исходное состояние проекта	2
4	Изменения, внесённые в репозиторий	3
4.1	Изменения в файле <code>src/main.cpp</code>	4
4.2	Изменения в файле <code>scripts/verify.py</code>	4
4.3	Добавление файла <code>scripts/run_experiments.py</code>	5
4.4	Изменения в <code>Makefile</code>	6
5	Описание реализованного алгоритма	6
6	Автоматизированная верификация результатов	6
7	Методика проведения экспериментов	7
8	Порядок запуска программы	8
9	Результаты вычислительных экспериментов	8
10	Анализ проделанной работы	9
11	Заключение	10
A	Фрагмент команды ограничения числа ядер в Linux	11

1 Цель работы

Целью лабораторной работы является модификация программы последовательного умножения квадратных матриц, разработанной в лабораторной работе №1, для параллельного выполнения с использованием технологии OpenMP, а также проведение серии вычислительных экспериментов с различным числом потоков и различными размерами входных данных.

2 Постановка задачи

Требовалось выполнить следующие действия:

1. модифицировать программу умножения квадратных матриц на языке C/C++ для параллельной работы по технологии OpenMP;
2. обеспечить чтение исходных матриц из файлов и запись результирующей матрицы в файл;
3. выводить время выполнения и объём вычислительной задачи;
4. реализовать автоматизированную верификацию результатов вычислений с использованием сторонней библиотеки или стороннего ПО;
5. провести серию экспериментов для размеров матриц примерно 200, 400, 800, 1200, 1600 и 2000 при различном количестве потоков (1, 2, 4, 8);
6. при наличии технической возможности исследовать влияние количества вычислительных ядер, в противном случае использовать фиксированное число доступных ядер.

3 Исходное состояние проекта

До начала выполнения лабораторной работы №2 репозиторий содержал реализацию последовательного умножения матриц и скрипт верификации на Python. Структура проекта имела следующий вид:

```
1 matrix-multiplication-lab/  
2 |-- data/
```

```
3 |    |-- matrix_a.txt
4 |    '-- matrix_b.txt
5 |-- scripts/
6 |    '-- verify.py
7 |-- src/
8 |    '-- main.cpp
9 |-- Makefile
10 |-- README.md
11 '-- report.pdf
```

В исходной версии программы:

- матрицы считывались из файлов `data/matrix_a.txt` и `data/matrix_b.txt`;
- умножение выполнялось последовательным тройным циклом;
- результат записывался в файл `data/result_cpp.txt`;
- выводилось только время выполнения и размер матрицы;
- верификация выполнялась отдельным скриптом `verify.py` с использованием библиотеки NumPy.

4 Изменения, внесённые в репозиторий

В рамках лабораторной работы №2 были модифицированы и добавлены следующие файлы:

- `src/main.cpp` — переработан для поддержки OpenMP;
- `scripts/verify.py` — доработан для более универсальной и автоматизированной проверки;
- `scripts/run_experiments.py` — добавлен для автоматического запуска серии экспериментов;
- `Makefile` — изменён для сборки программы с флагом OpenMP;
- `results/experiments.csv` — сформирован по итогам запуска серии экспериментов.

4.1 Изменения в файле src/main.cpp

Основные изменения в основной программе:

1. добавлено подключение заголовочного файла `omp.h`;
2. реализована возможность задания количества потоков через аргумент командной строки;
3. добавлено отключение динамического изменения числа потоков функцией `omp_set_dynamic(0)`;
4. внешний фрагмент вычислений распараллелен директивой OpenMP;
5. объём задачи вычисляется по формуле $2n^3 - n^2$;
6. улучшена обработка ошибок при чтении и записи файлов;
7. тип данных элементов матриц заменён на `long long` для снижения риска переполнения при больших размерах и значениях элементов.

Ключевой фрагмент распараллеливания представлен ниже:

```
1 #pragma omp parallel for collapse(2) schedule(static)
2 for (int i = 0; i < n; ++i) {
3     for (int j = 0; j < n; ++j) {
4         long long sum = 0;
5         for (int k = 0; k < n; ++k) {
6             sum += A.at(i, k) * B.at(k, j);
7         }
8         C.at(i, j) = sum;
9     }
10 }
```

Использование директивы `collapse(2)` позволяет объединить два внешних цикла по индексам `i` и `j` в одно пространство итераций, что улучшает распределение работы между потоками.

4.2 Изменения в файле scripts/verify.py

Скрипт верификации был расширен следующим образом:

- добавлена возможность передавать пути к входным и выходному файлам через аргументы командной строки;
- добавлен возврат кода завершения процесса: 0 при успешной верификации и 1 при несовпадении результатов;
- добавлен вывод максимального отклонения при обнаружении ошибки.

Идея верификации сохраняется прежней: эталонный результат вычисляется как произведение матриц средствами NumPy:

```

1 C_py = A @ B
2 if np.array_equal(C_cpp, C_py):
3     print("Verification successful")
4 else:
5     print("Results do not match")

```

Таким образом, требование автоматизированной проверки с использованием стороннего программного обеспечения выполнено.

4.3 Добавление файла `scripts/run_experiments.py`

Для автоматизации экспериментальной части в репозиторий был добавлен новый скрипт `scripts/run_experiments.py`. Его функции:

1. генерация случайных квадратных матриц заданных размеров;
2. запуск OpenMP-программы с разным количеством потоков;
3. автоматическая проверка корректности результата после каждого запуска;
4. вычисление ускорения и эффективности;
5. сохранение итоговых данных в файл `results/experiments.csv`.

Наличие отдельного сценария запуска экспериментов позволяет воспроизводимо получить таблицы для отчёта и упростить сравнительный анализ.

4.4 Изменения в Makefile

Для поддержки OpenMP в процессе сборки в `Makefile` добавлен флаг:

```
1 -fopenmp
```

Итоговая команда сборки имеет вид:

```
1 g++ -O2 -std=c++17 -fopenmp src/main.cpp -o matrix_mul_omp
```

5 Описание реализованного алгоритма

Алгоритм умножения матриц остаётся классическим. Для каждой ячейки результирующей матрицы C вычисляется сумма произведений элементов строки матрицы A и столбца матрицы B :

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}. \quad (1)$$

Общий объём вычислений для матриц размера $n \times n$ составляет:

$$N_{ops} = n^3 + (n^3 - n^2) = 2n^3 - n^2, \quad (2)$$

где первое слагаемое соответствует операциям умножения, а второе — операциям сложения.

Параллельная версия программы распределяет независимые вычисления элементов результирующей матрицы между несколькими потоками. Так как каждая пара индексов (i, j) вычисляется независимо, задача хорошо распараллеливается без необходимости синхронизации при записи результатов.

6 Автоматизированная верификация результатов

Для проверки корректности вычислений используется библиотека NumPy. После выполнения программы на C++ результирующая матрица, записанная в файл `data/result_cpp.txt`, считывается Python-скриптом и сравнивается с эталонным результатом, вычисленным оператором матричного умножения NumPy.

Данный подход удобен тем, что:

- NumPy реализует проверенную и широко используемую матричную арифметику;
- проверка не зависит от реализации на C++;
- верификация может выполняться автоматически после каждого экспериментального запуска.

В ходе серии экспериментов верификация была успешно пройдена для всех 24 запусков, что подтверждается значением `verified = yes` в итоговом CSV-файле.

7 Методика проведения экспериментов

Для оценки эффективности параллельной реализации была проведена серия вычислительных экспериментов при следующих параметрах:

- размеры матриц: 200, 400, 800, 1200, 1600, 2000;
- число потоков: 1, 2, 4, 8;
- число доступных процессоров в тестовой среде: 56.

Для каждой конфигурации измерялись:

- время выполнения T_p ;
- ускорение

$$S_p = \frac{T_1}{T_p}; \quad (3)$$

- эффективность

$$E_p = \frac{S_p}{p}. \quad (4)$$

Следует учитывать, что абсолютные значения времени зависят от конкретной вычислительной системы. В настоящем отчёте приведены результаты, полученные при запуске программы в доступной тестовой среде контейнера; на целевой машине пользователя значения могут немного отличаться.

8 Порядок запуска программы

Сборка проекта:

```
1 make
```

Ручной запуск программы для заданного числа потоков:

```
1 ./matrix_mul_omp data/matrix_a.txt data/matrix_b.txt data/  
  result_cpp.txt 4
```

Проверка результата:

```
1 python3 scripts/verify.py data/matrix_a.txt data/matrix_b.txt  
  data/result_cpp.txt
```

Автоматический запуск всей серии экспериментов:

```
1 python3 scripts/run_experiments.py
```

После выполнения сценария результаты сохраняются в файл `results/experiments`

9 Результаты вычислительных экспериментов

Ниже приведена заполненная таблица по данным файла `results/experiments.csv`

Таблица 1: Результаты вычислительных экспериментов

Размер n	Потоки p	Время T_p , с	Объём, опер.	Ускорение S_p	Эффективность E_p
200	1	0.007604	15960000	1.000000	1.000000
200	2	0.003770	15960000	2.016976	1.008488
200	4	0.001995	15960000	3.811529	0.952882
200	8	0.001036	15960000	7.339768	0.917471
400	1	0.085984	127840000	1.000000	1.000000
400	2	0.043234	127840000	1.988805	0.994403
400	4	0.021607	127840000	3.979451	0.994863
400	8	0.011085	127840000	7.756788	0.969599
800	1	0.814188	1023360000	1.000000	1.000000
800	2	0.427753	1023360000	1.903407	0.951703
800	4	0.368358	1023360000	2.210317	0.552579
800	8	0.373513	1023360000	2.179812	0.272476
1200	1	2.905375	3454560000	1.000000	1.000000

Размер n	Потоки p	Время T_p , с	Объём, опер.	Ускорение S_p	Эффективность E_p
1200	2	1.369779	3454560000	2.121054	1.060527
1200	4	1.353216	3454560000	2.147015	0.536754
1200	8	1.378809	3454560000	2.107163	0.263395
1600	1	9.753580	8189440000	1.000000	1.000000
1600	2	4.018162	8189440000	2.427374	1.213687
1600	4	4.531348	8189440000	2.152468	0.538117
1600	8	4.307801	8189440000	2.264167	0.283021
2000	1	52.252360	15996000000	1.000000	1.000000
2000	2	27.136160	15996000000	1.925562	0.962781
2000	4	26.935328	15996000000	1.939919	0.484980
2000	8	28.165695	15996000000	1.855177	0.231897

10 Анализ проделанной работы

По полученным данным видно, что параллельная реализация действительно сокращает время выполнения по сравнению с однопоточным вариантом, однако характер ускорения зависит от размера матрицы.

Для небольших матриц 200×200 и 400×400 наблюдается почти линейное ускорение вплоть до 8 потоков. Например, для размера 400 время уменьшилось с 0.085984 с до 0.011085 с, а ускорение составило 7.756788. Это объясняется тем, что задача ещё достаточно компактна и хорошо размещается в кэш-памяти, а накладные расходы OpenMP невелики.

Для матриц среднего и большого размера картина становится менее идеальной. Для $n = 800$ максимальное ускорение составило около 2.21, для $n = 1200$ — около 2.15, для $n = 1600$ — около 2.43, а для $n = 2000$ — около 1.94. При этом после 2–4 потоков дальнейшее увеличение числа потоков не даёт пропорционального выигрыша. На размере 2000 запуск с 8 потоками оказался даже медленнее, чем с 4 потоками.

Такое поведение связано с тем, что реализованный алгоритм является наивным тройным циклом без блочного разбиения. При росте размера матриц возрастает давление на подсистему памяти и пропускную способность кэшей, а обмен данными начинает ограничивать масштабирование сильнее, чем сами арифметические операции. Дополнительно на результат влияют накладные

расходы на создание и координацию потоков.

Значения эффективности выше единицы, встречающиеся для некоторых запусков с 2 потоками, объясняются погрешностью измерений, работой турборежима процессора и особенностями планировщика в тестовой среде. Такие значения допустимо интерпретировать как результат экспериментального шума на сравнительно коротких запусках.

В целом по результатам экспериментов можно сделать вывод, что модификация программы под OpenMP выполнена успешно: для всех протестированных размеров подтверждена корректность вычислений, а время выполнения по сравнению с однопоточной версией уменьшилось. Наиболее устойчивый практический выигрыш в данной реализации достигается в диапазоне 2–4 потоков.

11 Заключение

В лабораторной работе №2 была выполнена модификация программы умножения квадратных матриц на языке C++ для параллельного выполнения с использованием технологии OpenMP. В репозиторий были внесены изменения, обеспечивающие параллельное вычисление, управление числом потоков, вывод времени выполнения и объёма задачи, а также автоматическую проверку корректности результатов средствами Python и NumPy.

По итогам запуска серии экспериментов был сформирован файл `results/experiments` содержащий результаты для 24 конфигураций. Верификация прошла успешно во всех случаях. Полученные данные показали, что для небольших матриц достигается почти линейное ускорение, однако для размеров 800 и выше дальнейшее увеличение числа потоков после 2–4 начинает давать ограниченный эффект. Максимальный измеренный выигрыш на больших размерах был получен в диапазоне 2–4 потоков, а для матрицы 2000×2000 лучший результат составил 26.935328 с при 4 потоках против 52.252360 с при одном потоке.

Следовательно, цель лабораторной работы достигнута: последовательная программа успешно преобразована в параллельную OpenMP-версию, автоматизированы проверка и экспериментальный запуск, а также получены численные данные для анализа эффективности распараллеливания.

А Фрагмент команды ограничения числа ядер в Linux

При наличии необходимости число используемых ядер можно ограничивать системной утилитой `taskset`. Например:

```
1 taskset -c 0 ./matrix_mul_omp data/matrix_a.txt data/matrix_b.txt  
  data/result_cpp.txt 1  
2 taskset -c 0,1 ./matrix_mul_omp data/matrix_a.txt data/matrix_b.  
  txt data/result_cpp.txt 2  
3 taskset -c 0,1,2,3 ./matrix_mul_omp data/matrix_a.txt data/  
  matrix_b.txt data/result_cpp.txt 4
```